

Scenario 1: The Portfolio Dashboard

-- 1. User Net Worth (Current holdings valuation + cash)

Calculates U1 total value by adding his cash balance to the current market value of all his stocks.

```
SELECT u.Name, (ta.AvlBalance + COALESCE(SUM(ph.Current_Quantity_Held *  
md.DailyHigh), 0)) as Total_Net_Worth FROM AppUser u  
JOIN TradingAccount ta ON u.UserId = ta.UserId  
LEFT JOIN DematAccount da ON u.UserId = da.UserId  
LEFT JOIN Portfolio_Holds ph ON da.DematAccId = ph.DematAccId  
LEFT JOIN (SELECT DISTINCT ON (Ticker_Symbol) Ticker_Symbol, DailyHigh FROM  
MarketData ORDER BY Ticker_Symbol, TimeStamp DESC) md ON ph.Ticker_Symbol =  
md.Ticker_Symbol  
WHERE u.UserId = 'U1' GROUP BY u.Name, ta.AvlBalance;
```

	name character varying (100) 🔒	total_net_worth numeric 🔒
1	Amit	252250.00

-- 2. Portfolio Breakdown

Lists the specific companies and quantities of stocks held in the "D1" demat account.

```
SELECT fi.Ticker_Symbol, c.CompanyName, ph.Current_Quantity_Held  
FROM Portfolio_Holds ph  
JOIN Financial_Instruments fi ON ph.Ticker_Symbol = fi.Ticker_Symbol  
JOIN Company c ON fi.CompanyName = c.CompanyName  
WHERE ph.DematAccId = 'D1';
```

	ticker_symbol character varying (20) 🔒	companyname character varying (100) 🔒	current_quantity_held integer 🔒
1	TCS	TCS	10
2	RELI	Reliance	65
3	SUNP	SunPharma	2
4	TATM	TataMotors	3

-- 3. Sector Concentration

Counts how many different stocks account "D1" holds within each industry (e.g., Technology, Energy).

```

SELECT c.Sector, COUNT(fi.Ticker_Symbol) as Stock_Count
FROM Portfolio_Holds ph
JOIN Financial_Instruments fi ON ph.Ticker_Symbol = fi.Ticker_Symbol
JOIN Company c ON fi.CompanyName = c.CompanyName
WHERE ph.DematAccId = 'D1' GROUP BY c.Sector;

```

	sector character varying (50) 🔒	stock_count bigint 🔒
1	Automobile	1
2	Energy	1
3	Healthcare	1
4	Technology	1

-- 4. Inactive Investors

Identifies users who have a demat account but haven't actually bought any stocks yet.

```

SELECT u.Name FROM AppUser u
JOIN DematAccount da ON u.UserId = da.UserId
LEFT JOIN Portfolio_Holds ph ON da.DematAccId = ph.DematAccId
WHERE ph.Ticker_Symbol IS NULL;

```

	name character varying (100) 🔒
1	Sania
2	Jay

-- 5. High-Value Portfolios (>50 shares total)

Finds all demat accounts that hold a total of more than 50 shares across all their combined stocks.

```

SELECT DematAccId, SUM(Current_Quantity_Held) FROM Portfolio_Holds
GROUP BY DematAccId HAVING SUM(Current_Quantity_Held) > 50;

```

	demataccid character varying (50) 🔒	sum bigint 🔒
1	D1	80

-- 6. Diversification Check (>3 sectors)

Identifies "power users" who have diversified their investments across more than 3 different industry sectors.

```
SELECT da.UserId FROM Portfolio_Holds ph
JOIN Financial_Instruments fi ON ph.Ticker_Symbol = fi.Ticker_Symbol
JOIN Company c ON fi.CompanyName = c.CompanyName
JOIN DematAccount da ON ph.DematAccId = da.DematAccId
GROUP BY da.UserId HAVING COUNT(DISTINCT c.Sector) > 3;
```

	userid character varying (50) 🔒
1	U1

-- 7. Capital Distribution per Sector (Platform-wide)

Provides a platform-wide view of how many total shares are held by all users in each sector.

```
SELECT c.Sector, SUM(ph.Current_Quantity_Held) as Total_Quantity
FROM Portfolio_Holds ph
JOIN Financial_Instruments fi ON ph.Ticker_Symbol = fi.Ticker_Symbol
JOIN Company c ON fi.CompanyName = c.CompanyName
GROUP BY c.Sector;
```

	sector character varying (50) 🔒	total_quantity bigint 🔒
1	Technology	19
2	Automobile	3
3	Healthcare	2
4	Energy	78

-- 8. User Contact Directory (Verified)

Generates a mailing list of "Verified" users along with their current available trading cash.

```
SELECT u.Name, u.Email, ta.AvlBalance FROM AppUser u
JOIN TradingAccount ta ON u.UserId = ta.UserId
WHERE u.KycStatus = 'Verified';
```

	name character varying (100) 🔒	email character varying (100) 🔒	avlbalance numeric (15,2) 🔒
1	Amit	amit@mail.com	50000.00
2	Riya	riya@mail.com	30000.00
3	Raj	raj@mail.com	70000.00
4	Sania	sania@mail.com	0.00
5	Jay	null@mail.com	1000.00

-- 9. Sector Exposure (Technology % for U1)

Calculates what percentage of U1 total share count is concentrated specifically in the "Technology" sector.

```
SELECT (SUM(CASE WHEN c.Sector = 'Technology' THEN ph.Current_Quantity_Held ELSE
0 END) * 100.0 / SUM(ph.Current_Quantity_Held)) as Tech_Exposure_Pct
FROM Portfolio_Holds ph
JOIN Financial_Instruments fi ON ph.Ticker_Symbol = fi.Ticker_Symbol
JOIN Company c ON fi.CompanyName = c.CompanyName
WHERE ph.DematAccId = 'D1';
```

	tech_exposure_pct numeric 🔒
1	12.5000000000000000

-- 10. Holding Valuation (Live-ish)

Shows the current dollar/rupee value of every stock holding in the system based on the latest market highs.

```
SELECT ph.Ticker_Symbol, (ph.Current_Quantity_Held * md.DailyHigh) as Valuation
FROM Portfolio_Holds ph
JOIN (SELECT DISTINCT ON (Ticker_Symbol) Ticker_Symbol, DailyHigh
FROM MarketData ORDER BY Ticker_Symbol, TimeStamp DESC) md
ON ph.Ticker_Symbol = md.Ticker_Symbol;
```

	ticker_symbol character varying (20) 🔒	valuation numeric 🔒
1	TCS	35500.00
2	RELI	165750.00
3	SUNP	1000.00
4	RELI	20400.00
5	TCS	14200.00
6	INFY	8000.00
7	RELI	12750.00

-- 11. Depository Split

Shows the market share between different depositories (NSDL vs. CDSL) by counting linked accounts.

```
SELECT LinkedDepository, COUNT(*) FROM DematAccount
GROUP BY LinkedDepository;
```

	linkeddepository character varying (50) 🔒	count bigint 🔒
1	NSDL	4
2	CDSL	2

-- 12. High Balance / Unverified KYC

Flags users who have deposited money but haven't finished their legal verification—a potential compliance risk.

```
SELECT u.Name, ta.AvlBalance FROM AppUser u
JOIN TradingAccount ta ON u.UserId = ta.UserId
WHERE u.KycStatus != 'Verified' AND ta.AvlBalance > 0;
```

	name character varying (100) 🔒	avlbalance numeric (15,2) 🔒
1	Vikram	15000.00

-- 13. Consolidated Financial IDs

Creates a master list mapping every User ID to their specific Trading and Demat account numbers.

```

SELECT u.UserId, ta.TradingAccId, da.DematAccId
FROM AppUser u
LEFT JOIN TradingAccount ta ON u.UserId = ta.UserId
LEFT JOIN DematAccount da ON u.UserId = da.UserId;

```

	userid character varying (50) 🔒	tradingaccid character varying (50) 🔒	demataccid character varying (50) 🔒
1	U1	T1	D1
2	U2	T2	D2
3	U3	T3	D3
4	U4	T4	D4
5	U5	T5	D5
6	U6	T6	D6

-- 14. Simple Margin Calculation (20% of Portfolio Value)

Estimates a "borrowing limit" for Amit (U1) equal to 20% of his total portfolio's market value.

```

SELECT u.UserId, u.Name, SUM(ph.Current_Quantity_Held * md.DailyHigh) * 0.20 AS
Calculated_Margin
FROM AppUser u
JOIN DematAccount da ON u.UserId = da.UserId
JOIN Portfolio_Holds ph ON da.DematAccId = ph.DematAccId
JOIN ( SELECT DISTINCT ON (Ticker_Symbol) Ticker_Symbol, DailyHigh
FROM MarketData
ORDER BY Ticker_Symbol, TimeStamp DESC ) md ON ph.Ticker_Symbol =
md.Ticker_Symbol
WHERE u.UserId = 'U1' GROUP BY u.UserId, u.Name;

```

	userid [PK] character varying (50) ✎	name character varying (100) ✎	calculated_margin numeric 🔒
1	U1	Amit	40450.0000

-- Scenario 2: Trade Execution & Order Management

-- 1. Pending Sell Orders

Lists all "Sell" requests that have been placed but are not yet executed or completed.

SELECT * FROM Orders WHERE Order_Type = 'SELL' AND Status = 'Pending';

	order_id [PK] character varying (50)	order_type character varying (20)	price numeric (10,2)	status character varying (20)	orderqty integer	timestamp timestamp without time zone	tradingaccid character varying (50)	ticker_symbol character varying (20)
1	O5	SELL	3550.00	Pending	5	2026-04-12 14:41:52.599763	T1	TCS

-- 2. Trade-to-Order Mapping

Connects a specific order (O3) to its final execution details like price and exact time.

SELECT o.Order_ID, t.Execution_Price, t.Execution_Time
FROM Orders o JOIN Trade t ON o.Order_ID = t.Order_ID WHERE o.Order_ID = 'O3';

	order_id character varying (50)	execution_price numeric (10,2)	execution_time timestamp without time zone
1	O3	1485.00	2026-04-12 13:41:52.599763

-- 3. Slippage Analysis (Price paid > Price requested)

Finds "Buy" trades where the final price paid was higher than the user's initial requested price.

SELECT t.Trade_ID, o.Price as Requested, t.Execution_Price as Actual
FROM Trade t JOIN Orders o ON t.Order_ID = o.Order_ID
WHERE t.Execution_Price > o.Price AND o.Order_Type = 'BUY';

	trade_id character varying (50)	requested numeric (10,2)	actual numeric (10,2)
1	TR11	2600.00	2700.00

-- 4. High-Volume Trades

Filters for specific trade executions where more than 10 shares changed hands in a single go.

SELECT * FROM Trade WHERE Execution_Qty > 10;

	trade_id [PK] character varying (50)	execution_qty integer	execution_price numeric (10,2)	execution_time timestamp without time zone	order_id character varying (50)	demataccid character varying (50)
1	TR2	15	1485.00	2026-04-12 13:41:52.599763	O3	D3

-- 5. Daily Transaction Volume

Sums up the total amount of successful cash deposits and withdrawals made today.

SELECT SUM(Amount) FROM Financial_Transaction
WHERE Status = 'Success' AND TimeStamp::DATE = CURRENT_DATE;

	sum numeric
1	400.00

-- 6. Recent Activity for U2

Shows the 5 most recent stock trades performed by user Riya (U2).

```
SELECT t.* FROM Trade t
JOIN DematAccount da ON t.DematAccId = da.DematAccId
WHERE da.UserId = 'U2' ORDER BY Execution_Time DESC LIMIT 5;
```

	trade_id [PK] character varying (50)	execution_qty integer	execution_price numeric (10,2)	execution_time timestamp without time zone	order_id character varying (50)	demataccid character varying (50)
1	TR12	10	2800.00	2026-04-12 18:39:27.419124	O4	D2

-- 7. Order Distribution

Counts how many "Buy" vs. "Sell" orders exist in the entire system's history.

```
SELECT Order_Type, COUNT(*) FROM Orders GROUP BY Order_Type;
```

	order_type character varying (20)	count bigint
1	BUY	5
2	SELL	1

-- 8. Trade History with Tickers

Generates a log of every trade showing the specific stock symbol, quantity, and price.

```
SELECT t.Trade_ID, o.Ticker_Symbol, t.Execution_Qty, t.Execution_Price
FROM Trade t JOIN Orders o ON t.Order_ID = o.Order_ID;
```

	trade_id character varying (50)	ticker_symbol character varying (20)	execution_qty integer	execution_price numeric (10,2)
1	TR2	INFY	15	1485.00
2	TR11	RELI	5	2700.00
3	TR12	RELI	10	2800.00

-- 9. Average Buy Price for RELI (U2)

Calculates the average price Riya (U2) has paid for Reliance (RELI) across all her different buy orders.

```
SELECT AVG(Execution_Price) FROM Trade t
JOIN Orders o ON t.Order_ID = o.Order_ID
JOIN DematAccount da ON t.DematAccId = da.DematAccId
WHERE o.Ticker_Symbol = 'RELI' AND o.Order_Type = 'BUY' AND da.UserId = 'U2';
```

	avg numeric
1	2800.0000000000000000

-- 10. Settlement Audit

Maps every trade back to its demat account and stock ticker for auditing purposes.

```
SELECT t.Trade_ID, t.DematAccId, o.Ticker_Symbol
FROM Trade t JOIN Orders o ON t.Order_ID = o.Order_ID;
```

	trade_id character varying (50) 🔒	demataccid character varying (50) 🔒	ticker_symbol character varying (20) 🔒
1	TR2	D3	INFY
2	TR11	D1	RELI
3	TR12	D2	RELI

-- 11. Cash Flow Summary

Calculates the "Net Flow" (Deposits minus Withdrawals) for every account based only on successful transfers.

```
SELECT ft.TradingAccId,
SUM(CASE WHEN ft.Transaction_Type = 'Deposit' THEN ft.Amount ELSE -ft.Amount END)
as Net_Verified_Flow
FROM Financial_Transaction ft
JOIN TradingAccount ta ON ft.TradingAccId = ta.TradingAccId
JOIN LinkedBankAccount lb ON ft.BankAccountNo = lb.BankAccountNo
WHERE lb.UserId = ta.UserId -- Validation logic
AND ft.Status = 'Success'
GROUP BY ft.TradingAccId;
```

	tradingaccid character varying (50) 🔒	net_verified_flow numeric 🔒
1	T3	-400.00

-- 12. Largest Trade of the Day

Identifies the single most expensive trade (Quantity × Price) that occurred during the current day.

```
SELECT Trade_ID, (Execution_Qty * Execution_Price) as Volume_Value
FROM Trade
WHERE Execution_Time::DATE = CURRENT_DATE
ORDER BY Volume_Value DESC LIMIT 1;
```

	trade_id [PK] character varying (50)	volume_value numeric
1	TR12	28000.00

-- 13. SBI Transaction Logs

Specifically tracks all money transfers that moved through State Bank of India (SBI) accounts.

```
SELECT ft.* FROM Financial_Transaction ft
JOIN TradingAccount ta ON ft.TradingAccId = ta.TradingAccId
JOIN LinkedBankAccount lb ON ft.BankAccountNo = lb.BankAccountNo
WHERE lb.BankName = 'SBI' AND lb.UserId = ta.UserId;
```

	transaction_id [PK] character varying (50)	transaction_type character varying (50)	amount numeric (15,2)	payment_method character varying (50)	status character varying (20)	timestamp timestamp without time zone	tradingaccid character varying (50)	bankaccountno character varying
1	TX10	Withdraw	100.00	UPI	Success	2026-04-12 14:41:52.599763	T3	BA3
2	TX11	Withdraw	100.00	UPI	Success	2026-04-12 14:41:52.599763	T3	BA3
3	TX12	Withdraw	100.00	UPI	Success	2026-04-12 14:41:52.599763	T3	BA3
4	TX13	Withdraw	100.00	UPI	Success	2026-04-12 14:41:52.599763	T3	BA3

-- 14. Top Trading Tickers

Ranks the stocks on the platform by the frequency of trades (most popular to least popular).

```
SELECT o.Ticker_Symbol,
COUNT(t.Trade_ID) as Trade_Count FROM Orders o
JOIN Trade t ON o.Order_ID = t.Order_ID GROUP BY o.Ticker_Symbol
ORDER BY Trade_Count DESC;
```

	ticker_symbol character varying (20)	trade_count bigint
1	RELI	2
2	INFY	1

-- Scenario 3: Market Analytics & Watchlists

-- 1. Daily Gainers (>10% volatility)

Finds stocks that have seen high price swings (more than 10%) between their daily high and low.

```
SELECT Ticker_Symbol FROM MarketData
WHERE (DailyHigh - DailyLow) / DailyLow > 0.10;
```

	ticker_symbol character varying (20)
1	RELI

-- 2. Volume Spikes

Identifies stocks currently trading at volumes significantly higher than their own historical average.

```
SELECT m1.Ticker_Symbol, m1.Volume
FROM MarketData m1 WHERE m1.Volume > ( SELECT AVG(m2.Volume)
FROM MarketData m2 WHERE m1.Ticker_Symbol = m2.Ticker_Symbol );
```

	ticker_symbol character varying (20) 🔒	volume bigint 🔒
1	RELI	90000
2	TCS	150000
3	RELI	95000
4	INFY	80000
5	SUNP	210000
6	HDFC	90000

-- 3. Specific Watchlist Content

Lists all the stock symbols currently being tracked in the "Energy Focus" watchlist.

```
SELECT Ticker_Symbol FROM Watchlist_Contains wc
JOIN Watchlist w ON wc.Watchlist_ID = w.Watchlist_ID
WHERE w.Watchlist_Name = 'Energy Focus';
```

	ticker_symbol character varying (20) 🔒
1	RELI
2	ADANIG
3	ONGC

-- 4. Sector Performance (Avg Price)

Calculates the average trading price for all companies within specific sectors (e.g., Finance vs. Healthcare).

```
SELECT c.Sector, AVG(md.DailyHigh)
```

```
FROM MarketData md
JOIN Financial_Instruments fi ON md.Ticker_Symbol = fi.Ticker_Symbol
JOIN Company c ON fi.CompanyName = c.CompanyName GROUP BY c.Sector;
```

	sector character varying (50) 	avg numeric 
1	Technology	2575.0000000000000000
2	Finance	1500.0000000000000000
3	Healthcare	500.0000000000000000
4	Energy	2550.0000000000000000

-- 5. Market Volatility (Latest Spread)

Shows the "Spread" (difference between High and Low prices) for every stock based on the latest data.

```
SELECT DISTINCT ON (Ticker_Symbol)
    Ticker_Symbol,
    (DailyHigh - DailyLow) as Spread
FROM MarketData
ORDER BY Ticker_Symbol, TimeStamp DESC;
```

	ticker_symbol character varying (20) 	spread numeric 
1	HDFC	5.00
2	INFY	50.00
3	RELI	250.00
4	SUNP	20.00
5	TCS	250.00

-- 6. Empty Watchlists

Finds user-created watchlists that don't actually have any stocks saved in them yet.

```
SELECT Watchlist_ID, Watchlist_Name
FROM Watchlist WHERE Watchlist_ID NOT IN (SELECT Watchlist_ID
FROM Watchlist_Contains);
```

	watchlist_id [PK] character varying (50) 	watchlist_name character varying (100) 
1	W3	Empty Watchlist

-- 7. Historical Lows (7 days)

Finds the lowest price Infosys (INFY) has touched at any point in the last 7 days.

SELECT MIN(DailyLow) FROM MarketData

WHERE Ticker_Symbol = 'INFY' AND TimeStamp > NOW() - INTERVAL '7 days';

	min numeric 
1	1550.00

-- 8. NSE Stock Catalogz

Lists every company and ticker symbol specifically traded on the National Stock Exchange (NSE).

SELECT Ticker_Symbol, CompanyName FROM Financial_Instruments

WHERE Exchange = 'NSE' AND Instrument_Type = 'Stock';

	ticker_symbol [PK] character varying (20) 	companyname character varying (100) 
1	TCS	TCS
2	RELI	Reliance
3	INFY	Infosys
4	HDFC	HDFC
5	SUNP	SunPharma
6	TATM	TataMotors
7	ADANIG	AdaniGreen
8	ONGC	ONGC

-- 9. Max Volume Ticker

Identifies the single stock that has the highest trading activity (volume) in the database.

SELECT Ticker_Symbol

FROM MarketData ORDER BY Volume DESC LIMIT 1;

	ticker_symbol character varying (20)
1	SUNP

-- 10. Stable Stocks (<1% spread)

Filters for "quiet" stocks where the price hasn't fluctuated more than 1% during the day.

```
SELECT Ticker_Symbol FROM MarketData
WHERE (DailyHigh - DailyLow)/DailyHigh < 0.01;
```

	ticker_symbol character varying (20)
1	HDFC

-- 11. High Volume / Low Price

Flags "Penny Stocks" or cheaper stocks (under 3000) that are currently seeing massive trading activity.

```
SELECT * FROM MarketData WHERE Volume > 100000 AND DailyHigh < 3000;
```

	ticker_symbol [PK] character varying (20)	timestamp [PK] timestamp without time zone	dailyhigh numeric (10,2)	dailylow numeric (10,2)	curprice numeric (10,2)	volume bigint
1	SUNP	2026-04-12 14:41:52.599763	500.00	480.00	490.00	200000

-- 12. Exchange Distribution

Breaks down how many financial instruments are listed on the NSE versus other exchanges.

```
SELECT Exchange, COUNT(*) FROM Financial_Instruments GROUP BY Exchange;
```

	exchange character varying (20)	count bigint
1	NSE	8

-- 13. 7-Day History Export

Pulls a full chronological price and volume report for TCS over the past week for charting.

```
SELECT * FROM MarketData
WHERE Ticker_Symbol = 'TCS' AND TimeStamp > NOW() - INTERVAL '7 days'
ORDER BY TimeStamp DESC;
```

	ticker_symbol [PK] character varying (20)	timestamp [PK] timestamp without time zone	dailyhigh numeric (10,2)	dailylow numeric (10,2)	curprice numeric (10,2)	volume bigint
1	TCS	2026-04-12 14:41:52.599763	3550.00	3300.00	3450.00	120000